

Adaptive Spatio-Temporal Traffic Surveillance

Phase 2 — Model Training & Granular Helmet Compliance

Milestone	Phase 2 of 4 (Days 6–12): Model Training & Helmet Compliance
Author	Harsh Pandhe
Repository	https://github.com/harsh-pandhe/traffic-surveillance-system
Wheel model	YOLOv8-cls selected — 86.1% accuracy (4 classes, 3 architectures compared)
Helmet model	YOLOv8n — mAP@50 0.764 (7 classes)

Executive summary. Phase 2 delivers trained models on real data. For wheel-count classification, three architectures are benchmarked under an identical, **leak-free** protocol across four classes (2/3/4/6+ wheelers): a custom SmallCNN baseline (0.787), RTMDet's CSPNeXt backbone (0.695), and YOLOv8-cls (0.861, selected). For helmet compliance, a YOLOv8n detector reaches mAP@50 0.764 on a **held-out test split** never used for model selection. All metrics are recomputed from the datasets and persisted in results/phase2/; the pretraining asymmetry between the arms is stated explicitly rather than glossed over.

Correction notice. An earlier version of this report gave 0.751 / 0.683 / 0.867 for the three architectures. That run split the dataset by *crop* rather than by source image, so several vehicles from the same photo appeared on both sides of the split (76% / 91% / 65% of 2-, 4- and 6+-wheeler crops came from such photos), and the 3-wheeler class used whole photos while every other class used tight crops. Both defects are fixed and the numbers above are the corrected ones. The model ranking is unchanged.

1. Wheel-Count Classification — CNN vs RTMDet vs YOLO

Four classes: 2-Wheeler (bicycle/motorcycle, COCO), 3-Wheeler (auto-rickshaw crops), 4-Wheeler (car), 6+ Wheeler (bus/truck). Balanced ~308–350 train images per class, 96×96 crops, CPU training.

Model	Acc	Macro-P	Macro-R	Macro-F1	Lat (ms)	Params	Pretrained
SmallCNN	0.787	0.756	0.753	0.753	1.1	0.24M	scratch
RTMDet-CSPNeXt	0.695	0.638	0.645	0.639	1.0	2.35M	scratch
YOLOv8-cls (selected)	0.861	0.829	0.826	0.827	2.9	—	ImageNet

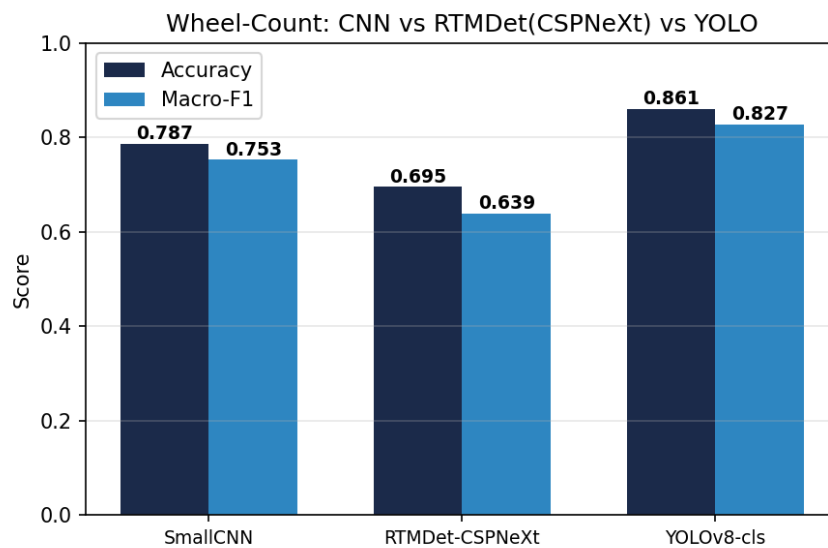


Figure 1. Accuracy and macro-F1 across the three architectures.

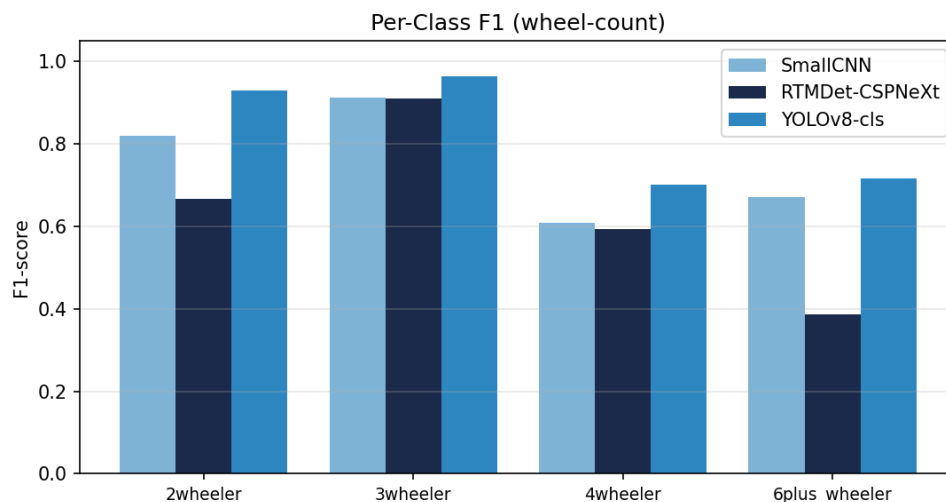


Figure 2. Per-class F1 on the leak-free split. 3-Wheeler remains strongest (0.96) even after cropping; car vs bus/truck is now the hard pair.

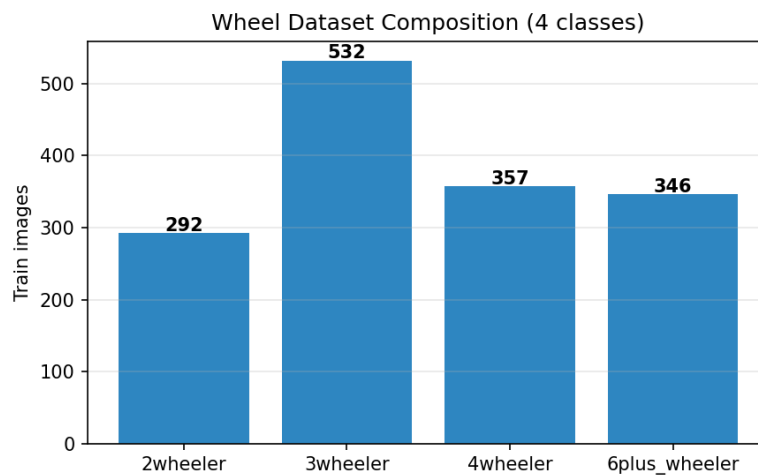


Figure 3. Balanced 4-class wheel dataset.

Methodological note

This comparison is **not pretraining-neutral**. YOLOv8-cls starts from ImageNet weights; SmallCNN and CSPNeXt train from scratch, so part of YOLO's margin is transfer learning rather than architecture. CSPNeXt (2.35M params) has the least data per parameter and underfits at this dataset size, yet it is the **fastest at inference** (1.0 ms — essentially tied with the 10x smaller SmallCNN and ~3x faster than YOLOv8-cls), because the depthwise 5x5 CSP design is CPU-efficient. On the earlier dataset it was still improving with longer training (0.562 at 12 epochs to 0.683 at 40), so its figure here is a floor, not a ceiling. For this dataset size the pretrained YOLOv8-cls is the correct production choice.

RTMDet implementation

mmdetection/mmcv (RTMDet's official home) will not install here: openmim fails on Python 3.12 (pkgutil.ImpImporter removed) and mmcv ships no wheels for torch 2.12. RTMDet's architectural contribution — the CSPNeXt backbone — is therefore reimplemented directly in PyTorch (`src/models/cspnext.py`) following the paper: depthwise 5x5 CSP blocks, channel attention, SiLU, SPPF, RTMDet-tiny scaling.

2. Granular Helmet Compliance — YOLOv8n

7-class rider/helmet detection (driver/passenger × helmet/no-helmet + bike), 368 train / 65 val / 52 test, YOLO format. Fine-tuned YOLOv8n, 25 epochs, 416×416, CPU. The figures below are on the **held-out test split**; the validation split was used for epoch selection and is therefore optimistically biased. Both splits are small, so both carry wide confidence intervals — they corroborate each other at roughly 0.73-0.76 rather than one superseding the other.

Metric	Value
mAP@50	0.764
mAP@50-95	0.372
Precision	0.732
Recall	0.746

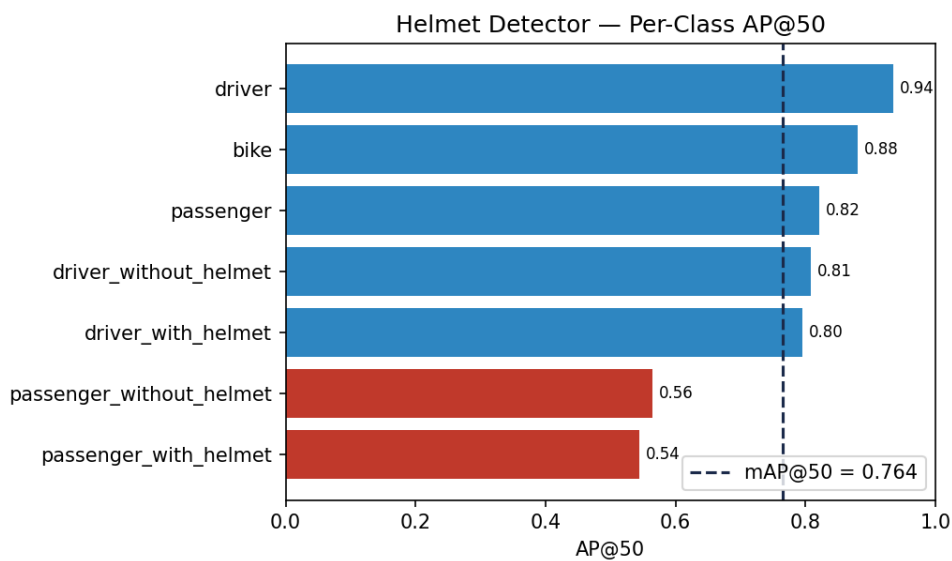


Figure 4. Per-class AP@50 (red = below the mAP line).

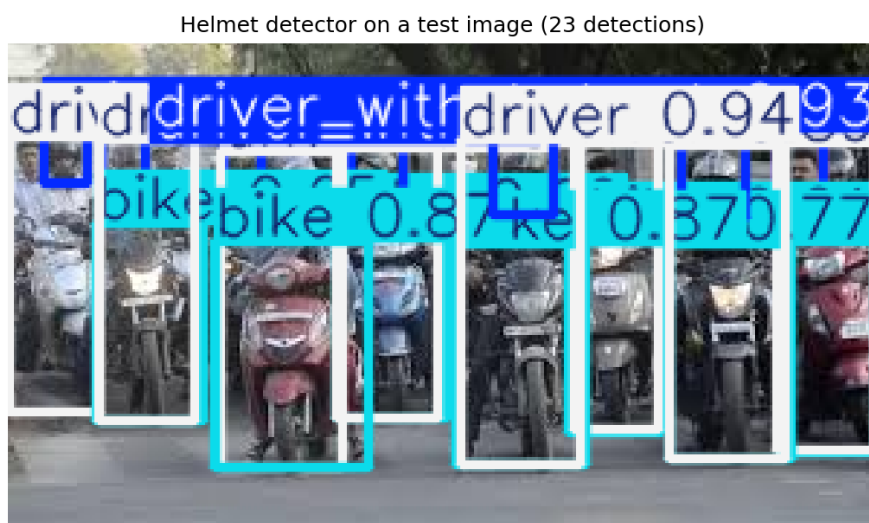


Figure 5. Trained detector on a real test image — riders, bikes, and helmet status.

Passenger-helmet classes are weakest (fewest instances, small objects) — tracked as follow-up issue #8.

3. Reproduce

Command	Purpose
python scripts/build_wheel_dataset.py	Build 4-class wheel crops
python scripts/train_wheel_classifier.py	Train + benchmark CNN vs YOLO
python scripts/train_helmet_detector.py --data ...	Train + eval helmet detector
python scripts/make_phase2_assets.py	Regenerate this report's figures

4. Status & Follow-ups

Deliverable	Status
Baseline CNN trained	Complete
YOLO model trained (wheel + helmet)	Complete
P/R/mAP comparison + selected model	Complete
3-Wheeler class (auto-rickshaw)	Complete (issue closed)
Granular helmet detector (7-class mAP)	Complete
Low AP passenger-helmet classes	Open (issue #8)

Conclusion. Phase 2 is complete: both models trained and benchmarked on real data, all four wheel classes active, and the granular helmet detector reporting per-class AP. Every figure is reproducible from the committed code.